

Agenda de contactos de la Quantum Wallet

Operaciones CRUD sobre SQLite integradas a Quantum Core

Implementación de la tabla contactos y de las operaciones de creación, lectura, actualización y eliminación desde Python, con validación en dos capas, controles de seguridad e integración con las transferencias de la billetera

ESTUDIANTE	Deibis Zuluaga Baena
NÚCLEO	Fundamentos de Software · FUNSO_2_N_H
ACTIVIDAD	Operaciones CRUD (Create, Read, Update, Delete) en Quantum Core
HERRAMIENTAS	SQLite 3 · Python 3.14 (sqlite3, logging) · DB Browser for SQLite
ENTREGABLES	practica_crud.py, quantum_wallet.db, crud_contactos.log, diagrama de integración (PNG, SVG y .drawio) e informe
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
FECHA DE ENTREGA	4 de octubre de 2026

15 páginas · 11 figuras

Objetivo del trabajo

Objetivo: realizar las operaciones CRUD (Create, Read, Update, Delete) sobre una nueva tabla de Quantum Core, `contactos`, vinculada a los usuarios mediante una llave foránea, sin alterar la estructura principal de la base de datos, e implementar cada operación como una función del script `practica_crud.py`.

CÓDIGO ORIGINAL DE LA ACTIVIDAD

El enunciado de la actividad y el repositorio del curso definen la tabla `contactos` (identificador, apodo, número y llave foránea al usuario) y cuatro operaciones: insertar cinco contactos para el usuario 1, mostrar en consola sus contactos, cambiar el apodo de un contacto mediante una sentencia con `WHERE` y eliminar el contacto con identificador 2. Esos requisitos se conservan completos. Para enriquecer el ejercicio se adicionaron campos, restricciones, controles de seguridad y funciones complementarias, identificados en el código con el comentario `[AGREGADO]`.

BASE DE DATOS DE PARTIDA

La práctica opera sobre `quantum_wallet.db`, generada por `configurar_db.py` con las tablas de usuarios, billeteras y transacciones de Quantum Core. Para que el proyecto pueda ejecutarse de forma independiente, la carpeta incluye una copia de ese script. La tabla `contactos` se agrega sin modificar las tablas existentes.

ALCANCE DE LA SOLUCIÓN

Además de las cuatro operaciones solicitadas, la solución incorpora validación en Python y en la base de datos, autorización por usuario en las actualizaciones y eliminaciones, una lista blanca de campos editables, búsqueda de contactos, marcación de favoritos, transferencias de dinero a un contacto que tiene cuenta en Quantum Wallet, registro de operaciones en consola y en archivo, y un conjunto de pruebas que comprueba que las operaciones inválidas son rechazadas.

Estructura de la tabla contactos

La tabla almacena la agenda de cada usuario. Las filas con fondo amarillo corresponden a los campos adicionados para enriquecer el ejercicio.

CAMPO	TIPO	RESTRICCIONES	PROPÓSITO
id_contacto	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único del contacto.
id_usuario	INTEGER	NOT NULL; FK → usuarios ON DELETE CASCADE	Dueño de la agenda.
apodo	TEXT	NOT NULL; COLLATE NOCASE; 1 a 40 caracteres; único por agenda	Nombre con el que el usuario identifica al contacto.
numero	TEXT	NOT NULL; solo dígitos; 3 a 15 caracteres; único por agenda	Número telefónico del contacto.
email	TEXT	CHECK: formato de correo	Correo electrónico opcional.
id_usuario_contacto	INTEGER	FK → usuarios ON DELETE SET NULL; distinto de id_usuario	Cuenta en Quantum Wallet del contacto, cuando la tiene.
favorito	INTEGER	NOT NULL; DEFAULT 0; CHECK: 0 o 1	Marca de contacto frecuente.
fecha_creacion	TEXT	NOT NULL; DEFAULT fecha actual	Fecha de registro del contacto.
fecha_actualizacion	TEXT	Se asigna en cada actualización	Trazabilidad de la última modificación.

Decisiones de diseño

DECISIÓN	JUSTIFICACIÓN
Número como texto	El número es un identificador, no una cantidad: se almacena como <code>TEXT</code> para conservar ceros iniciales y admitir líneas cortas como la de emergencias (123). La restricción <code>NOT GLOB '[^0-9]*'</code> exige que contenga únicamente dígitos, y el límite de 15 corresponde a la longitud máxima de un número telefónico internacional.
Unicidad por agenda	Las restricciones <code>UNIQUE (id_usuario, numero)</code> y <code>UNIQUE (id_usuario, apodo)</code> impiden duplicados dentro de una misma agenda, pero permiten que dos usuarios registren el mismo número.
Apodo sin distinción de mayúsculas	Con <code>COLLATE NOCASE</code> , «Mamita» y «MAMITA» se consideran el mismo apodo. Esto evita ambigüedades al buscar un contacto o transferirle dinero por su apodo.
Dos llaves foráneas hacia usuarios	<code>id_usuario</code> indica de quién es la agenda; <code>id_usuario_contacto</code> indica si el contacto también es cliente de Quantum Wallet. La segunda es opcional, porque no todos los contactos tienen cuenta.
Creación limpia de la tabla	El script elimina y vuelve a crear la tabla en cada ejecución. Al eliminarla, SQLite reinicia también su contador <code>AUTOINCREMENT</code> , de modo que el contacto 2 corresponde siempre al mismo registro y las operaciones del enunciado son reproducibles.
Índice por usuario	<code>idx_contactos_usuario</code> acelera la consulta más frecuente: leer la agenda de un usuario.

Relación de la tabla contactos con el sistema

Quantum Wallet · Integración de la tabla contactos

Relación de la agenda de contactos con usuarios, billeteras y transacciones de quantum_wallet.db

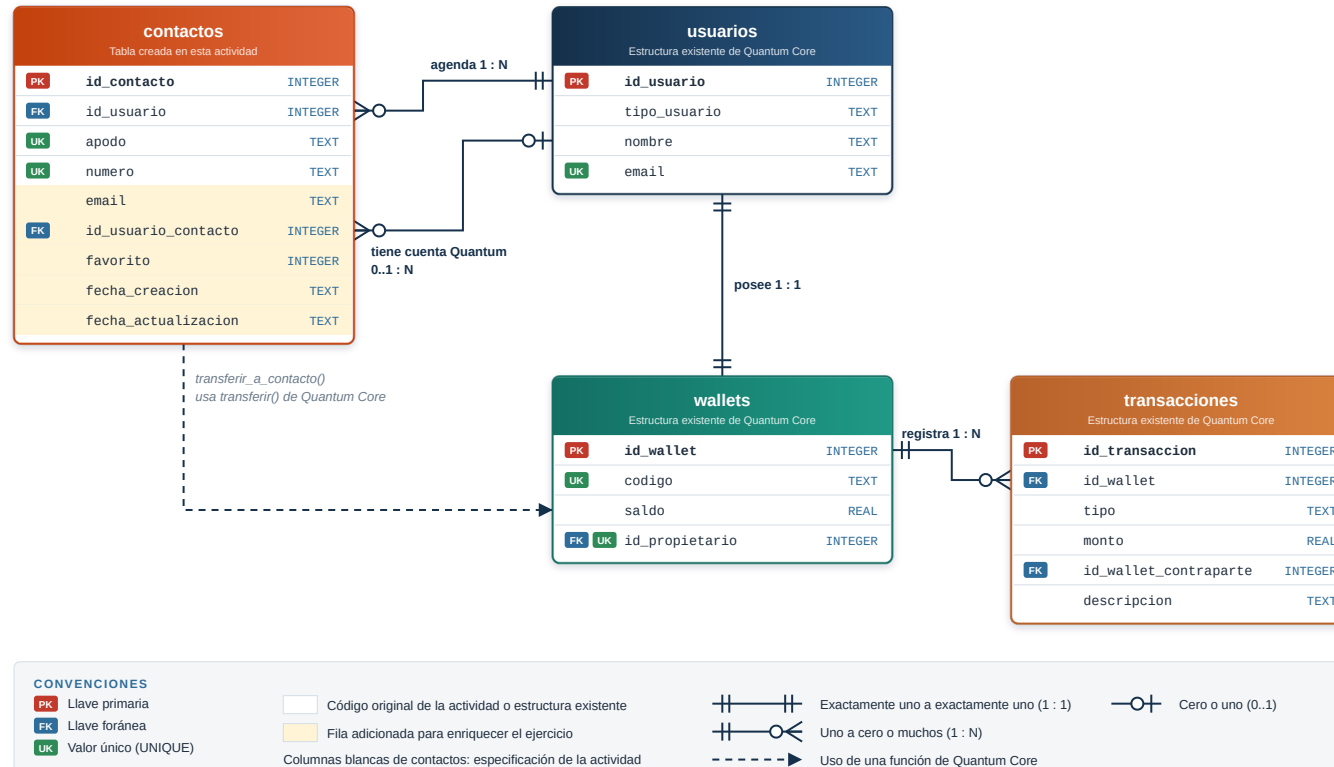


Figura 1. Integración de la tabla contactos con usuarios, billeteras y transacciones, en notación de pata de gallo. Elaborado en formato .drawio, editable con Draw.io Integration para Visual Studio Code.

Comportamiento de las relaciones

La tabla se integra a Quantum Core mediante dos llaves foráneas hacia `usuarios` y mediante la reutilización de las funciones del sistema para mover dinero.

RELACIÓN	CARDINALIDAD	COMPORTAMIENTO
usuarios – contactos (agenda)	1 : N	Cada contacto pertenece a un usuario. Si el usuario se elimina, su agenda se elimina con él (<code>ON DELETE CASCADE</code>).
usuarios – contactos (cuenta Quantum)	0..1 : N	Un contacto puede corresponder a otro cliente de Quantum Wallet. Si ese cliente se elimina, el contacto se conserva y la referencia queda nula (<code>ON DELETE SET NULL</code>).
contactos – wallets – transacciones	—	La función <code>transferir_a_contacto()</code> ubica al contacto por su apodo y ejecuta <code>transferir()</code> de <code>configurar_db.py</code> . La transferencia queda registrada en <code>transacciones</code> y respeta las reglas del sistema: saldo suficiente, límite por transacción y atomicidad del débito y el crédito.

INTEGRACIÓN SIN DUPLICAR REGLAS

La agenda no maneja dinero por su cuenta: delega la transferencia en la misma función que usa el resto del sistema. Así, una transferencia hecha desde la agenda queda sujeta a las restricciones `CHECK`, a los triggers y a la conciliación de saldos que ya protegen a Quantum Core.

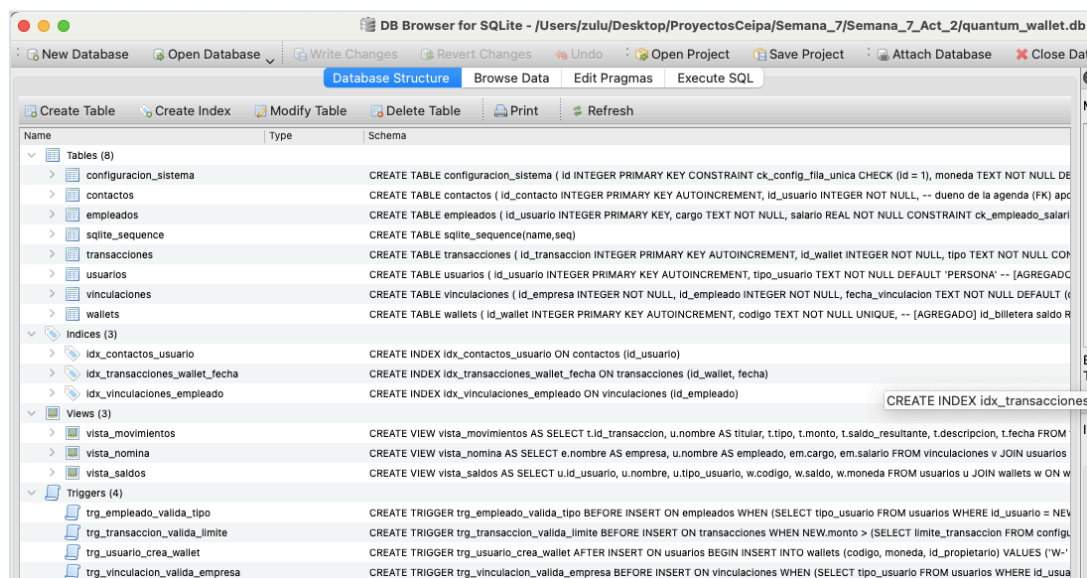


Figura 2. Estructura de `quantum_wallet.db` en DB Browser for SQLite: la tabla `contactos` y su índice se suman a las tablas existentes de Quantum Core.

Las operaciones CRUD en practica_crud.py

Cada operación del enunciado se implementó en una función propia y documentada. Todas usan parámetros `?` y se ejecutan dentro de una transacción.

OPERACIÓN	FUNCIÓN	SENTENCIA SQL	CONTROL APLICADO
C · Create	<code>crear_contactos()</code>	<code>INSERT</code> con <code>executemany</code>	Valida todas las filas antes de insertar; las guarda en una sola transacción: entran todas o ninguna.
R · Read	<code>leer_contactos()</code>	<code>SELECT ... LEFT JOIN usuarios ... WHERE id_usuario = ?</code>	Devuelve solo la agenda del usuario indicado y agrega el nombre del contacto en Quantum Wallet cuando tiene cuenta.
U · Update	<code>actualizar_apodo()</code>	<code>UPDATE ... WHERE id_contacto = ? AND id_usuario = ?</code>	Modifica una sola fila, solo si pertenece al usuario, y verifica que la sentencia haya afectado exactamente un registro.
D · Delete	<code>borrar_contacto()</code>	<code>DELETE ... WHERE id_contacto = ? AND id_usuario = ?</code>	Comprueba que el contacto exista y pertenezca al usuario; elimina una fila, nunca la tabla, y registra qué se eliminó.

Funciones adicionales

FUNCIÓN	PROPÓSITO
<code>actualizar_contacto()</code>	Actualiza varios campos a la vez, solo si están en la lista blanca <code>CAMPOS_EDITABLES</code> .
<code>buscar_contactos()</code>	Busca por apodo o número con <code>LIKE</code> ; escapa los comodines <code>%</code> y <code>_</code> que escriba el usuario.
<code>transferir_a_contacto()</code>	Transfiere dinero a un contacto con cuenta en Quantum Wallet, por su apodo.
<code>validar_apodo()</code> , <code>validar_numero()</code> , <code>validar_email()</code> , <code>validar_id()</code>	Primera capa de validación, previa a cualquier sentencia SQL.
<code>configurar_log()</code>	Registra cada operación en la consola y en <code>crud_contactos.log</code> .
<code>probar_seguridad()</code>	Ejecuta las pruebas de inyección SQL y de operaciones inválidas.

Actualización y eliminación con WHERE

```
# ----- U : UPDATE -----
with conexion:
    cursor = conexion.execute(
        "UPDATE contactos SET apodo = ?, fecha_actualizacion = datetime('now', 'localtime') "
        "WHERE id_contacto = ? AND id_usuario = ?;",
        (nuevo_apodo, id_contacto, id_usuario),
    )
    if cursor.rowcount == 0:
        raise LookupError(f"el contacto {id_contacto} no existe en la agenda del usuario {id_usuario}")

# ----- D : DELETE -----
with conexion:
    cursor = conexion.execute(
        "DELETE FROM contactos WHERE id_contacto = ? AND id_usuario = ?;",
        (id_contacto, id_usuario),
    )
```

La condición combina la llave primaria con el dueño de la agenda. La primera parte garantiza que se afecta una sola fila; la segunda impide que un usuario modifique o elimine contactos de otro. Una sentencia `UPDATE` o `DELETE` sin `WHERE` afectaría a todas las filas de la tabla.

Controles sobre la tabla contactos

Las reglas se aplican en dos capas: Python valida los datos antes de construir la sentencia, y la base de datos las vuelve a exigir con restricciones. Si un programa omite la validación de Python e inserta directamente con SQL, la base de datos rechaza igualmente los datos inválidos.

REGLA	PRIMERA CAPA: PYTHON	SEGUNDA CAPA: BASE DE DATOS
Apodo de 1 a 40 caracteres	<code>validar_apodo()</code>	<code>CHECK ck_contacto_apodo</code>
Número solo con dígitos	<code>validar_numero()</code>	<code>CHECK ck_contacto_numero</code>
Correo con formato válido	<code>validar_email()</code>	<code>CHECK ck_contacto_email</code>
Sin duplicados en la agenda	—	<code>UNIQUE uq_contacto_numero , uq_contacto_apodo</code>
Usuario existente	—	Llave foránea con <code>PRAGMA foreign_keys = ON</code>
No agendarse a sí mismo	—	<code>CHECK ck_contacto_no_propio</code>

Controles de seguridad

CONTROL	RIESGO QUE MITIGA
Consultas parametrizadas (?)	Inyección SQL: los datos del usuario se transmiten como valores y nunca se interpretan como código.
WHERE con llave primaria y dueño	Modificación masiva por error y acceso a contactos de otro usuario.
Verificación de filas afectadas	Operaciones que no encuentran el registro y fallan en silencio.
Lista blanca de campos	Cambios en columnas protegidas, como el dueño de la agenda, a través de la actualización genérica.
Identificadores enteros	Uso de textos arbitrarios como identificador en las operaciones de escritura.
Escape de comodines en LIKE	Búsquedas que devuelven más resultados de los esperados.
Transacciones atómicas	Cargas parciales o datos inconsistentes ante un error intermedio.
Registro de operaciones	Falta de trazabilidad sobre lo que se creó, modificó o eliminó.

Pruebas de inyección SQL

La lectura se ejecutó con el texto `1 OR 1=1` como identificador de usuario, y la búsqueda con `' OR '1'='1`. Ambas devolvieron cero filas: SQLite trató cada texto como un valor por comparar y no como parte de la sentencia, por lo que la condición no se alteró.

Operaciones inválidas

El script intenta once operaciones que violan las reglas. Todas fueron rechazadas y revertidas.

OPERACIÓN INTENTADA	CAPA	RESULTADO
Número con letras (300-ABC)	Python	<code>ValueError</code> : número no válido
Apodo vacío	Python	<code>ValueError</code> : longitud del apodo
Número repetido en la agenda	Base de datos	<code>UNIQUE constraint failed</code>
Apodo repetido con otras mayúsculas	Base de datos	<code>UNIQUE constraint failed</code>
Contacto para un usuario inexistente	Base de datos	<code>FOREIGN KEY constraint failed</code>
Agendarse a sí mismo	Base de datos	<code>CHECK constraint failed: ck_contacto_no_propio</code>
Modificar un contacto de otro usuario	Autorización	<code>LookupError</code> : el contacto no está en la agenda
Borrar un contacto que ya no existe	Autorización	<code>LookupError</code>
Actualizar el dueño de la agenda	Lista blanca	<code>ValueError</code> : campo no permitido
SQL directo, sin validación de Python	Base de datos	<code>CHECK constraint failed: ck_contacto_numero</code>
Transferir a un contacto sin cuenta	Integración	<code>ValueError</code> : no tiene cuenta en Quantum Wallet

Evidencias de las operaciones CRUD

Después de crear la base con `configurar_db.py`, el script se ejecutó con `python3 practica_crud.py --pausas`. La opción `--pausas` detiene la ejecución después de cada operación de escritura, de modo que el estado de la tabla puede verificarse en DB Browser for SQLite antes y después de cada cambio.

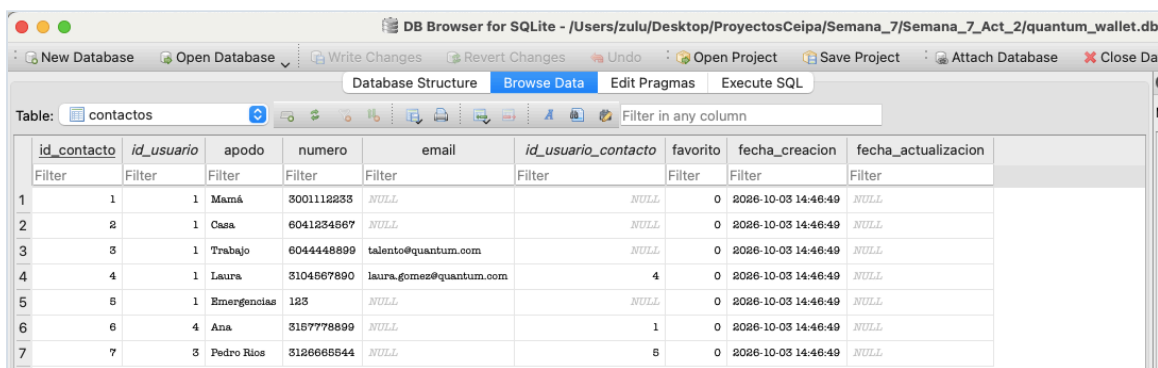
C · Create

```

QUANTUM CORE - Operaciones CRUD sobre la tabla contactos
14:46:49 | INFO | Tabla contactos creada (PK, 2 FK, 5 CHECK, 2 UNIQUE, 1 indice)
----- C - CREATE -----
14:46:49 | INFO | C - Create: 5 contactos insertados para el usuario 1
14:46:49 | INFO | C - Create: 1 contacto insertado para el usuario 4
14:46:49 | INFO | C - Create: 1 contacto insertado para el usuario 3
>>> Captura C: en DB Browser abra la tabla contactos (Browse Data, Refresh).
>>> Presione Enter para continuar...

```

Figura 3. Consola: creación de la tabla e inserción de cinco contactos para el usuario 1 y de un contacto para cada uno de los usuarios 4 y 3.



The screenshot shows the DB Browser for SQLite interface. The 'contactos' table is selected, and the 'Browse Data' tab is active. The table contains 7 records. The columns are: id_contacto, id_usuario, apodo, numero, email, id_usuario_contacto, favorito, fecha_creacion, and fecha_actualizacion. The data is as follows:

id_contacto	id_usuario	apodo	numero	email	id_usuario_contacto	favorito	fecha_creacion	fecha_actualizacion
1	1	Mamá	3001112233	NULL	NULL	0	2026-10-03 14:46:49	NULL
2	2	Casa	6041234567	NULL	NULL	0	2026-10-03 14:46:49	NULL
3	3	Trabajo	6044448899	talento@quantum.com	NULL	0	2026-10-03 14:46:49	NULL
4	4	Laura	3104567890	laura.gomez@quantum.com	4	0	2026-10-03 14:46:49	NULL
5	5	Emergencias	123	NULL	NULL	0	2026-10-03 14:46:49	NULL
6	6	Ana	3157778899	NULL	1	0	2026-10-03 14:46:49	NULL
7	7	Pedro Ríos	3126665544	NULL	5	0	2026-10-03 14:46:49	NULL

Figura 4. Tabla contactos después del Create: siete registros, con «Mamá» en el identificador 1 y «Casa» en el identificador 2.

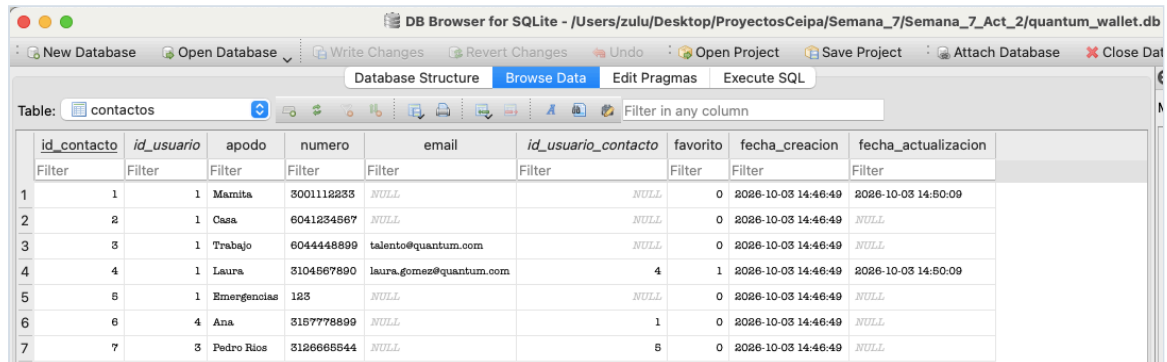
R · Read y U · Update

```
--- R - READ ---
14:50:09 | INFO | R - Read: 5 contactos del usuario 1
14:50:09 | INFO | 1 Mamá 3001112233 cuenta Quantum: -
14:50:09 | INFO | 2 Casa 6041234567 cuenta Quantum: -
14:50:09 | INFO | 3 Trabajo 6044448899 cuenta Quantum: -
14:50:09 | INFO | 4 Laura 3104567890 cuenta Quantum: Laura Gomez
14:50:09 | INFO | 5 Emergencias 123 cuenta Quantum: -

--- U - UPDATE ---
14:50:09 | INFO | U - Update: contacto 1 'Mamá' -> 'Mamita' (1 fila modificada)
14:50:09 | INFO | U - Update: contacto 4 actualizado {'favorito': 1}

>>> Captura U: actualice DB Browser (Refresh) y vea el apodo 'Mamita'.
>>> Presione Enter para continuar...
```

Figura 5. Consola: lectura de los cinco contactos del usuario 1, cambio del apodo «Mamá» por «Mamita» y marcación de Laura como favorita.



The screenshot shows the DB Browser for SQLite interface. The 'contactos' table is selected, and the 'Browse Data' tab is active. The table contains 7 records. The first record (id_contacto 1) has been updated from 'Mamá' to 'Mamita'. The fourth record (id_contacto 4) has been updated from 'Laura' to 'Laura' with 'favorito' set to 1. The table structure is as follows:

id_contacto	id_usuario	apodo	numero	email	id_usuario_contacto	favorito	fecha_creacion	fecha_actualizacion
1	1	Mamita	3001112233	NULL	NULL	0	2026-10-03 14:46:49	2026-10-03 14:50:09
2	2	Casa	6041234567	NULL	NULL	0	2026-10-03 14:46:49	NULL
3	3	Trabajo	6044448899	talento@quantum.com	NULL	0	2026-10-03 14:46:49	NULL
4	4	Laura	3104567890	laura.gomez@quantum.com	4	1	2026-10-03 14:46:49	2026-10-03 14:50:09
5	5	Emergencias	123	NULL	NULL	0	2026-10-03 14:46:49	NULL
6	6	Ana	3187778899	NULL	1	0	2026-10-03 14:46:49	NULL
7	7	Pedro Rios	3126665544	NULL	5	0	2026-10-03 14:46:49	NULL

Figura 6. Tabla contactos después del Update: el identificador 1 cambió a «Mamita», Laura tiene favorito = 1 y ambos registros muestran su fecha de actualización.

D · Delete

```
--- D - DELETE ---
14:52:51 | INFO | D - Delete: contacto 2 ('Casa', 6041234567) eliminado (1 fila)

>>> Captura D: actualice DB Browser (Refresh): el contacto 2 ya no esta.
>>> Presione Enter para continuar...
```

Figura 7. Consola: eliminación del contacto 2 («Casa», 6041234567).

DB Browser for SQLite - /Users/zulu/Desktop/ProyectosCeipa/Semana_7/Semana_7_Act_2/quantum_wallet.db

Database Structure Browse Data Edit Pragma Execute SQL

Table: contactos

id_contacto	id_usuario	apodo	numero	email	id_usuario_contacto	favorito	fecha_creacion	fecha_actualizacion
Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	1 Mamita	3001112233	NULL	NULL	0	2026-10-03 14:46:49	2026-10-03 14:50:09
2	3	1 Trabajo	6044448899	talento@quantum.com	NULL	0	2026-10-03 14:46:49	NULL
3	4	1 Laura	3104567890	laura.gomez@quantum.com	4	1	2026-10-03 14:46:49	2026-10-03 14:50:09
4	6	1 Emergencias	123	NULL	NULL	0	2026-10-03 14:46:49	NULL
5	6	4 Ana	3167778899	NULL	1	0	2026-10-03 14:46:49	NULL
6	7	3 Pedro Rios	3126666644	NULL	6	0	2026-10-03 14:46:49	NULL

Figura 8. Tabla contactos después del Delete: seis registros; el identificador 2 ya no existe.

Funcionalidades adicionales, integración y pruebas

```

--- BUSQUEDA [AGREGADO] ---
14:53:45 | INFO | Busqueda 'la' en la agenda del usuario 1: 1 resultado(s) ['Laura']

--- INTEGRACION CON QUANTUM CORE [AGREGADO] ---
14:53:45 | INFO | Transferencia de 50,000.00 a 'Laura' registrada en Quantum Core
14:53:45 | INFO | Saldo de Ana Torres : 220,000.50 -> 170,000.50
14:53:45 | INFO | Saldo de Laura Gomez : 4,349,999.50 -> 4,399,999.50

--- SEGURIDAD Y VALIDACION [AGREGADO] ---
14:53:45 | INFO | Inyeccion SQL en la lectura: leer_contactos(conexion, '1 OR 1=1')
14:53:45 | INFO | R - Read: 0 contactos del usuario 1 OR 1=1
14:53:45 | INFO | -> 0 filas: el texto se trato como un valor, no como codigo SQL
14:53:45 | INFO | Busqueda '' OR '1'='1' en la agenda del usuario 1: 0 resultado(s) []
14:53:45 | INFO | -> la busqueda con comillas devolvio 0 filas
14:53:45 | WARNING | RECHAZADA Numero con letras -> ValueError: numero no valido '300-ABC': solo digitos, entre 3 y 15
14:53:45 | WARNING | RECHAZADA Apodo vacio -> ValueError: el apodo debe tener entre 1 y 40 caracteres
14:53:45 | WARNING | RECHAZADA Numero repetido en la agenda -> IntegrityError: UNIQUE constraint failed: contactos.id_usuario, contac
tos.numero
14:53:45 | WARNING | RECHAZADA Apodo repetido (sin importar mayusculas) -> IntegrityError: UNIQUE constraint failed: contactos.id_usu
ario, contactos.apodo
14:53:45 | WARNING | RECHAZADA Contacto para un usuario inexistente -> IntegrityError: FOREIGN KEY constraint failed
14:53:45 | WARNING | RECHAZADA Agendarse a si mismo -> IntegrityError: CHECK constraint failed: ck_contacto_no_propio
14:53:45 | WARNING | RECHAZADA Modificar un contacto de otro usuario -> LookupError: el contacto 6 no existe en la agenda del usuario
1
14:53:45 | WARNING | RECHAZADA Borrar un contacto que ya no existe -> LookupError: el contacto 2 no existe en la agenda del usuario 1
14:53:45 | WARNING | RECHAZADA Actualizar un campo fuera de la lista blanca -> ValueError: campos no permitidos: ['id_usuario']
14:53:45 | WARNING | RECHAZADA SQL directo que evita la validacion de Python -> IntegrityError: CHECK constraint failed: ck_contacto_
numero
14:53:45 | WARNING | RECHAZADA Transferir a un contacto sin cuenta Quantum -> ValueError: el contacto 'Mamita' no tiene cuenta en Qua
ntum Wallet
14:53:45 | INFO | Resultado: 11 de 11 operaciones invalidas rechazadas

--- ESTADO FINAL ---
14:53:45 | INFO | R - Read: 4 contactos del usuario 1
14:53:45 | INFO | 1 Mamita 3001112233 cuenta Quantum: -
14:53:45 | INFO | 3 Trabajo 6044448899 cuenta Quantum: -
14:53:45 | INFO | * 4 Laura 3104567890 cuenta Quantum: Laura Gomez
14:53:45 | INFO | 5 Emergencias 123 cuenta Quantum: -

Registro completo en crud_contactos.log
o zulu@MacBook-Air-de-Deibis Semana_7_Act_2 %

```

Figura 9. Consola: búsqueda, transferencia de 50.000 a Laura, pruebas de seguridad (11 de 11 rechazadas) y estado final de la agenda.

id_transaccion	id_wallet	tipo	monto	saldo_resultante	id_wallet_contraparte	descripcion	fecha
1	1	RECARGA	100000.0	100000.0	NULL	Recarga	2026-10-03 14:45:14
2	2	PAGO	30000.0	70000.0	NULL	Pago en comercio	2026-10-03 14:45:14
3	3	RECARGA	800000.0	800000.0	NULL	Recarga	2026-10-03 14:45:14
4	4	PAGO	200000.0	300000.0	NULL	Pago a proveedor	2026-10-03 14:45:14
5	5	RECARGA	1000000.0	1000000.0	NULL	Capital de trabajo	2026-10-03 14:45:14
6	6	TRANSFERENCIA_ENVIADA	4800000.0	5800000.0	4	Pago de nomina	2026-10-03 14:45:14
7	7	TRANSFERENCIA_RECIBIDA	4800000.0	4800000.0	3	Pago de nomina	2026-10-03 14:45:14
8	8	TRANSFERENCIA_ENVIADA	3200000.0	2300000.0	5	Pago de nomina	2026-10-03 14:45:14
9	9	TRANSFERENCIA_RECIBIDA	3200000.0	3200000.0	3	Pago de nomina	2026-10-03 14:45:14
10	10	TRANSFERENCIA_ENVIADA	180000.5	4349999.5	1	Devolucion de prestamo	2026-10-03 14:45:15
11	11	TRANSFERENCIA_RECIBIDA	180000.5	280000.5	4	Devolucion de prestamo	2026-10-03 14:45:15
12	13	TRANSFERENCIA_ENVIADA	80000.0	170000.5	4	Pago de almuerzo (Laura)	2026-10-03 14:53:45
13	14	TRANSFERENCIA_RECIBIDA	50000.0	4399999.5	1	Pago de almuerzo (Laura)	2026-10-03 14:53:45

Figura 10. Tabla transacciones: la transferencia a Laura quedó registrada como débito en la billetera de Ana Torres y crédito en la de Laura Gomez (identificadores 13 y 14).

INTERPRETACIÓN

La transferencia redujo el saldo de Ana Torres de 220.000,50 a 170.000,50 e incrementó el de Laura Gomez de 4.349.999,50 a 4.399.999,50. El identificador de las transacciones pasa de 11 a 13 porque el número 12 lo utilizó el usuario temporal de la prueba de borrado en cascada de configurar_db.py ; AUTOINCREMENT no reutiliza identificadores.

```

Semana_7 > Semana_7_Act_2 > crud_contactos.log
Semana_7 > Semana_7_Act_2 > crud_contactos.log
1 2026-10-03 14:46:49 INFO Tabla contactos creada (PK, 2 FK, 5 CHECK, 2 UNIQUE, 1 indice)
2 2026-10-03 14:46:49 INFO C - Create: 5 contactos insertados para el usuario 1
3 2026-10-03 14:46:49 INFO C - Create: 1 contacto insertado para el usuario 4
4 2026-10-03 14:46:49 INFO C - Create: 1 contacto insertado para el usuario 3
5 2026-10-03 14:50:09 INFO R - Read: 5 contactos del usuario 1
6 2026-10-03 14:50:09 INFO 1 Mamá 300112233 cuenta Quantum: -
7 2026-10-03 14:50:09 INFO 2 Casa 6041234567 cuenta Quantum: -
8 2026-10-03 14:50:09 INFO 3 Trabajo 604448899 cuenta Quantum: -
9 2026-10-03 14:50:09 INFO 4 Laura 3104567890 cuenta Quantum: Laura Gomez
10 2026-10-03 14:50:09 INFO 5 Emergencias 123 cuenta Quantum: -
11 2026-10-03 14:50:09 INFO U - Update: contacto 1 'Mamá' -> 'Mamita' (1 fila modificada)
12 2026-10-03 14:50:09 INFO U - Update: contacto 4 actualizado {'favorito': 1}
13 2026-10-03 14:52:51 INFO D - Delete: contacto 2 ('Casa', 6041234567) eliminado (1 fila)
14 2026-10-03 14:53:45 INFO Busqueda 'la' en la agenda del usuario 1: 1 resultado(s) ['Laura']
15 2026-10-03 14:53:45 INFO Transferencia de 50,000.00 a 'Laura' registrada en Quantum Core
16 2026-10-03 14:53:45 INFO Saldo de Ana Torres : 220,000.50 -> 170,000.50
17 2026-10-03 14:53:45 INFO Saldo de Laura Gomez : 4,349,999.50 -> 4,399,999.50
18 2026-10-03 14:53:45 INFO Inyeccion SQL en la lectura: leer_contactos(conexion, '1 OR 1=1')
19 2026-10-03 14:53:45 INFO R - Read: 0 contactos del usuario 1 OR 1=1
20 2026-10-03 14:53:45 INFO -> 0 filas: el texto se trato como un valor, no como codigo SQL
21 2026-10-03 14:53:45 INFO Busqueda '' OR '1'='1' en la agenda del usuario 1: 0 resultado(s) []
22 2026-10-03 14:53:45 INFO -> la busqueda con comillas devolvio 0 filas
23 2026-10-03 14:53:45 WARNING RECHAZADA Numero con letras -> ValueError: numero no valido '300-ABC': solo digitos, entre 3 y 15
24 2026-10-03 14:53:45 WARNING RECHAZADA Apodo vacio -> ValueError: el apodo debe tener entre 1 y 40 caracteres
25 2026-10-03 14:53:45 WARNING RECHAZADA Numero repetido en la agenda -> IntegrityError: UNIQUE constraint failed: contactos.id_usuario, 4
26 2026-10-03 14:53:45 WARNING RECHAZADA Apodo repetido (sin importar mayusculas) -> IntegrityError: UNIQUE constraint failed: contactos.id_usuario, 4
27 2026-10-03 14:53:45 WARNING RECHAZADA Contacto para un usuario inexistente -> IntegrityError: FOREIGN KEY constraint failed
28 2026-10-03 14:53:45 WARNING RECHAZADA Agendarse a si mismo -> IntegrityError: CHECK constraint failed: ck_contacto_no_agenda
29 2026-10-03 14:53:45 WARNING RECHAZADA Modificar un contacto de otro usuario -> LookupError: el contacto 6 no existe en la agenda del usu
30 2026-10-03 14:53:45 WARNING RECHAZADA Borrar un contacto que ya no existe -> LookupError: el contacto 2 no existe en la agenda del usu
31 2026-10-03 14:53:45 WARNING RECHAZADA Actualizar un campo fuera de la lista blanca -> ValueError: campos no permitidos: {'id_usuario': 1}
32 2026-10-03 14:53:45 WARNING RECHAZADA SQL directo que evita la validacion de Python -> IntegrityError: CHECK constraint failed: ck_cont
33 2026-10-03 14:53:45 WARNING RECHAZADA Transferir a un contacto sin cuenta Quantum -> ValueError: el contacto 'Mamita' no tiene cuenta q
34 2026-10-03 14:53:45 INFO Resultado: 11 de 11 operaciones invalidas rechazadas
35 2026-10-03 14:53:45 INFO R - Read: 4 contactos del usuario 1
36 2026-10-03 14:53:45 INFO 1 Mamita 300112233 cuenta Quantum: -
37 2026-10-03 14:53:45 INFO 3 Trabajo 604448899 cuenta Quantum: -
38 2026-10-03 14:53:45 INFO * 4 Laura 3104567890 cuenta Quantum: Laura Gomez
  
```

Figura 11. Archivo crud_contactos.log en Visual Studio Code: registro de lo leído (R), lo actualizado (U), lo eliminado (D) y de cada operación rechazada.

Conclusiones y oportunidades de mejora

Las cuatro operaciones del enunciado se cumplieron y quedaron verificadas. Se crearon cinco contactos para el usuario 1, se leyeron en consola, el apodo «Mamá» cambió a «Mamita» y el contacto 2 fue eliminado. Las capturas del antes y el después y el registro de la consola lo confirman.

La cláusula WHERE es la principal protección de una operación de escritura. Al combinar la llave primaria con el dueño de la agenda, cada actualización o eliminación afecta una sola fila y solo dentro de la agenda que corresponde.

La validación en dos capas hace que la base de datos se proteja a sí misma. Python rechaza los datos inválidos antes de enviarlos, y las restricciones de la tabla rechazan incluso las sentencias que omiten esa validación.

La agenda amplía la funcionalidad de Quantum Core sin duplicar reglas. Las transferencias a un contacto se ejecutan con las mismas funciones del sistema y quedan sujetas a sus controles de saldo, límite y atomicidad.

Oportunidades de mejora

MEJORA	DESCRIPCIÓN
Eliminación lógica	Marcar los contactos como inactivos en lugar de borrarlos, para permitir su recuperación y conservar el historial de transferencias asociadas.
Importación de contactos	Reutilizar el proceso de carga masiva desde Excel para incorporar agendas completas con la misma validación.
Interfaz de usuario	Exponer las funciones CRUD en un menú de consola o una interfaz web para su uso por parte de los clientes.

REPOSITORIO

El código fuente, la base de datos, el registro de operaciones, el diagrama de integración (fuente .drawio y exportaciones PNG y SVG), las capturas, el README y este informe se encuentran en github.com/DavisZulu/Semana_7_Act_2.

Referencias

- Castro, E. y de Pablo, C. (2013). *Desarrollo de bases de datos: casos prácticos*. RA-MA.
- DB Browser for SQLite. (2026). *DB Browser for SQLite* [Software]. <https://sqlitebrowser.org>
- Hueso, L. (2012). *Gestión de bases de datos* (2.ª ed.). RA-MA.
- OWASP Foundation. (2026). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Peláez Loaiza, S. (2026). *FUNDAMENTOS-DE-SOFTWARE* [Repositorio de código]. GitHub. <https://github.com/SimonP8/FUNDAMENTOS-DE-SOFTWARE>
- Python Software Foundation. (2026a). *sqlite3: DB-API 2.0 interface for SQLite databases*. The Python Standard Library. <https://docs.python.org/3/library/sqlite3.html>
- Python Software Foundation. (2026b). *logging: Logging facility for Python*. The Python Standard Library. <https://docs.python.org/3/library/logging.html>
- SQLite. (2026a). *CREATE TABLE*. https://www.sqlite.org/lang_createtable.html
- SQLite. (2026b). *SQLite foreign key support*. <https://www.sqlite.org/foreignkeys.html>
- SQLite. (2026c). *SQL language expressions*. https://www.sqlite.org/lang_expr.html
- W3Schools. (2026). *SQL UPDATE statement*. https://www.w3schools.com/sql/sql_update.asp